Troubleshooting Journal

Data and analytics work at Nexora: pandas, PostgreSQL, Metabase and Excel

Volume 1 · October to November 2026

Days logged	Entries	Errors	Wrong results	How-tos
5	67	17	17	33

Contents

How this journal works	Structure, the log card fields and the daily routine
Index	Every entry on one page: search by tool, type or tag
Patterns	Lessons that came up more than once
Day pages	29 October 2026: 13 entries · 30 October 2026: 21 entries · 31 October 2026: 12 entries · 2 November 2026: 10 entries · 4 November 2026 (covers 3 to 4 Nov): 11 entries
Templates	A blank day page and log card to copy for each new day

How this journal works

Every problem you solve becomes one log card, filed under the day it happened. The Index lets you find any card by tool, type or tag, and the Patterns page collects the lessons that keep coming back.

The four parts

Part	What it holds	Update it
Index	One line per card: ID, tool, type, title, tags	After every new card
Patterns	Lessons seen on two or more cards, with what to do	Weekly review
Day pages	Date, focus, outcome, then the day's log cards	During and at the end of each day
Templates	A blank day page and log card	Copy one for each new day

The log card

Field	What to write
ID	Date plus a running number: `20261029-05` is the 5th card on 29 Oct 2026. Sorts by date automatically.
Tool	pandas, SQL, Excel, Python, Metabase, Bash.
Type	**Error**: a message appeared. **Wrong result**: no error, but the output was wrong. **How-to**: a new task you worked out.
Tags	2 to 4 search words: the function or concept, e.g. `ROW_NUMBER`, merge, dates.
Problem	Exactly what you saw. Paste the error text word for word, so you can search for it later.
Cause	Why it happened, in one or two sentences.
Fix	The code that worked, trimmed to the lines that matter.
Why it works	The two to four details you would forget in a month.
Lesson	One line you could apply to a different problem.

The routine

1. **As soon as a problem is solved**, write its card. Five minutes now saves an hour of re-asking later.
2. **At the end of the day**, fill the day's Focus and Outcome lines and add the cards to the Index.
3. **Once a week**, scan the Index. Any lesson that appears on two cards goes on the Patterns page.
4. **Once a month**, start a new volume. Carry the Index and Patterns forward.
5. **Before asking for help**, search the Index for the error text or a tag. The answer may already be here.

Keeping it searchable. Paste error messages exactly, including quotes and capitals. Searching this PDF (Ctrl+F) for `does not exist` or `has no attribute` finds every related card.

Index

One line per log card, oldest first. Types: **Error** = a message appeared, **Wrong result** = wrong output with no error, **How-to** = a new task.

ID	Tool	Type	Title	Tags
20261029-01	pandas	How-to	Name the dialer output file with the Start Time date range	dates, f-string, files
20261029-02	pandas	Error	Splitting Caller Ids: [1] vs .str[1], and "'DataFrame' object has no attribute 'str'"	strings, duplicate columns, .str
20261029-03	pandas	How-to	Extract customer and agent_id from Contact Info with regex	regex, str.extract, nullable ints
20261029-04	pandas	How-to	Combine Call Date and a 12-hour Call Time into one datetime	dates, parsing, 12-hour clock
20261029-05	SQL	Wrong result	Leads whose last 5 calls are all one disposition	ROW_NUMBER, HAVING, BOOL_AND
20261029-06	SQL	Wrong result	Raw verification query showed non-Zero dispositions	window functions, WINDOW, verification
20261029-07	SQL	How-to	Latest disposition is Zero, plus the distribution of the last 5	FILTER, STRING_AGG, two-level aggregation
20261029-08	SQL	How-to	Unreachable list, name normalisation, and last-10 pivot with call times	regexp_replace, VALUES, pivot
20261029-09	SQL	Error	column ol.client_id does not exist	Metabase, column names
20261029-10	SQL	Error	syntax error at or near "=" (Position 5128)	JOIN syntax, aliases
20261029-11	SQL	Wrong result	Leads repeated after joining customer_status_history	one-to-many join, DISTINCT ON, ARRAY_AGG
20261029-12	pandas	Error	TypeError: Can only merge Series or DataFrame objects, a <class 'str'> was passed	merge, latest row per key
20261029-13	Excel	How-to	Tag Active-but-unreachable clients as automatic traders	IF, COUNTIF, confidence levels
20261030-01	pandas	How-to	Select ledger rows for yesterday	dates, half-open range, sort_values
20261030-02	Polars	Wrong result	The combined ledger seemed to stop at the 27th	ledger_loader, diagnostics, Excel-saved CSV
20261030-03	Excel	Wrong result	SUMIFS returned 0 for negative balance protection	SUMIFS, $ references, COUNTIF
20261030-04	Excel	How-to	Extract the number from BR575346: ****	REGEXEXTRACT, MID, FIND
20261030-05	Excel	Error	The formula shows as text instead of calculating	cell format, Show Formulas
20261030-06	Excel	Wrong result	UNIQUE(FILTER(..., OR(...))) doesn't filter	FILTER, OR logic, arrays
20261030-07	pandas	Error	KeyError: 'ClientID' in groupby	variable names, columns
20261030-08	pandas	How-to	Fill Zynixx Balance and Last Trade with .map()	map, lookup, set_index

ID	Tool	Type	Title	Tags
20261030-09	pandas	Wrong result	Latest balance taken from a random row	drop_duplicates, latest row per key
20261030-10	pandas	Error	KeyError: Index([Ellipsis], dtype='object')	placeholder code
20261030-11	pandas	How-to	Add a result sheet to the original workbook	ExcelWriter, openpyxl
20261030-12	pandas	Error	Combining every file in a folder	os.listdir, concat, built-in names
20261030-13	pandas	Error	DtypeWarning: Columns (21: expiryDate) have mixed types	dtype, read_csv, mixed types
20261030-14	pandas	Error	MemoryError: Unable to allocate 487 MiB ... shape (1, 63807297)	memory, usecols, concat
20261030-15	Polars	Error	'LazyFrame' object has no attribute 'to_pandas'	LazyFrame, collect
20261030-16	Polars	Error	Kernel crashed on lf.collect()	memory, streaming, sink
20261030-17	Polars	How-to	Append the ark folder (CSV and XLSX) to the transactions	read_excel, diagonal_relaxed, source_file
20261030-18	Polars	How-to	transactions_loader.py: a module that can't crash the kernel	module, select before collect
20261030-19	pandas	Wrong result	Aligning ark columns gave four amount columns	rename, normalised names, clash check
20261030-20	pandas	How-to	Match the main file's columns and types exactly (minus id)	reindex, Int64, dtypes
20261030-21	pandas	Wrong result	Times show as 59:48.4 (date and hour lost)	Excel, CSV, data loss
20261031-01	Excel	Error	FILTER rejected: 'BILL' in single quotes	quotes, FILTER, UNIQUE
20261031-02	pandas	How-to	Organised, faster Stringee CDR combine script	pathlib, calamine, xlsxwriter, # %% cells
20261031-03	pandas	How-to	Read IDs as text; convert dates and durations after	dtype, to_timedelta, [h]:mm:ss
20261031-04	pandas	Wrong result	Old dialer notebook: Uno start times lost and durations zeroed	regression test, Uno, timedelta
20261031-05	pandas	How-to	Call gaps with groupby().shift() instead of a loop	groupby, shift, vectorise
20261031-06	pandas	Error	Grouper for 'Date' not 1-dimensional	pivot_table, keys
20261031-07	pandas	How-to	Multi-day reports with checks that fail loudly	concat, drop_duplicates, consistency check
20261031-08	pandas	Wrong result	Report showed only the latest date: wrong input folder	inputs, folders, debugging order
20261031-09	pandas	How-to	Parse 10/28/2026 09:34:25 PM safely	to_datetime, format, month-first
20261031-10	pandas	How-to	Use the newest team list in a folder	pathlib, st_mtime
20261031-11	Python	Error	KeyError on os.environ: token used as the variable name	environment variables, setx, secrets

ID	Tool	Type	Title	Tags
20261031-12	pandas	Wrong result	Over 31 trading days in a month	nunique, normalize, assert
20261102-01	SQL	How-to	Lead source for depositing customers	DISTINCT ON, self-join, COALESCE
20261102-02	pandas	How-to	Flag which customers traded in October	isin, np.where, Period
20261102-03	pandas	How-to	Total and traded clients per Manager and agent	groupby, nunique, where, subtotals
20261102-04	SQL	Wrong result	Calls after conversion would repeat per lead	fan-out, MIN, FILTER, STRING_AGG
20261102-05	SQL	How-to	Start from the pasted ID list so no ID disappears	UNNEST, ARRAY, LEFT JOIN, check_status
20261102-06	SQL	How-to	Join a Metabase saved question safely	{{#3268}}, DISTINCT ON, team list
20261102-07	SQL	How-to	Keep only the Sales team's calls	JOIN vs LEFT JOIN, filter on joined table
20261102-08	SQL	How-to	30-day counts beside the day buckets	FILTER, INTERVAL, cross-check
20261102-09	Process	Wrong result	A 30-day request rebuilt three times	requirements, output columns
20261102-10	Metabase	How-to	Admin IDs that share an IP	Summarize, distinct count, STRING_AGG, private IPs
20261104-01	pandas	How-to	When a lambda is fine, and when it's 900x slower	apply, vectorise, benchmark
20261104-02	pandas	How-to	Deposits and withdrawals net of cancellations	where, groupby, ledger types
20261104-03	pandas	Error	np.where got one argument: a bracket closed too late	np.where, &, brackets
20261104-04	pandas	Wrong result	Latest balance taken from a random row (again)	sort_values, drop_duplicates, repeated
20261104-05	pandas	Wrong result	Campaign flag: 'deposited on' vs 'first deposit was'	first row per group, requirements
20261104-06	pandas	How-to	Trading days and their spread per client	nunique, join per group, to_period
20261104-07	pandas	How-to	Excel export in the MIS design	openpyxl, ExcelWriter, number_format
20261104-08	pandas	Wrong result	NBP always 0: rows filtered away one step early	filter order, isin
20261104-09	Python	Error	TypeError: insert expected 2 arguments, got 1	sys.path, imports, PYTHONPATH
20261104-10	Python	How-to	Writing functions in three levels	def, parameters, defaults, None, raise
20261104-11	pandas	How-to	Prove a rewrite didn't change the numbers	assert_frame_equal, testing

Patterns

Lessons that showed up on more than one card. Read this page first when something looks wrong but no error appears.

Pattern	Seen in	What to do
Sort by the date before keep='first'	20261030-01 20261030-09 20261104-04	'Latest row per key' needs sort_values(date, ascending=False) right before drop_duplicates(keep='first').
Check the inputs before the code	20261030-02 20261031-08 20261104-08	Print which files and rows are used (value_counts after each filter) before changing any logic.
Repeated rows mean a one-to-many join or duplicate keys	20261029-11 20261029-12 20261030-21 20261102-01 20261102-04 20261102-06	Collapse the many side to one row per key first. Check COUNT(*) = COUNT(DISTINCT key), validate='many_to_one', or duplicated(subset=key).
Agree the output before building	20261102-09 20261104-05	Name the columns to add, what to keep, and the exact rule (first / any / only).
Check what is really stored before filtering	20261029-05 20261029-08 20261029-09 20261030-03 20261030-07 20261102-07	Look at real values and column names: df.columns, SELECT DISTINCT, value_counts, COUNTIF with wildcards.
Start from the thing every row must represent	20261102-05	Every ID must appear? Start from the IDs and LEFT JOIN the rest; label why a value is zero.
Duplicate or double-used names break things	20261029-02 20261030-19 20261031-06	Two columns with one name, or one column as both pivot key and value. Check names after every rename.
Names, brackets and arguments	20261029-01 20261030-12 20261031-11 20261104-03 20261104-09	Built-in names, a token's value used as its name, a bracket closing late, a missing argument.
Verify against raw rows or an old version	20261029-06 20261029-07 20261029-13 20261030-02 20261031-04 20261104-11	Compare counts with raw data; run old and new code on the same input; assert_frame_equal.
Build checks that fail loudly	20261031-07 20261031-09 20261031-12 20261102-05 20261102-08	assert impossible values, raise on unreadable dates, cross-check two outputs, label every zero.
Prefer whole-column code to row loops	20261031-05 20261104-01 20261104-02	Column operators, np.where/select, groupby built-ins first; apply(lambda) only when nothing else fits.
Load only what you need	20261030-14 20261030-15 20261030-16 20261030-18	Estimate rows x columns x 8 bytes. Select columns and filter rows before collecting; sink big outputs to disk.

Pattern	Seen in	What to do

Text that looks like a date or number isn't one	`20261029-04`
	`20261029-12`
	`20261030-13`
	`20261030-20`
	`20261031-03`
	`20261031-09`
		Read IDs as text; give date formats explicitly; convert with to_datetime / to_timedelta / Int64.

Excel changes data when it saves	`20261030-02`
	`20261030-05`
	`20261030-21`
		Never save a CSV from Excel. Long IDs become 2.61029E+17 and times become 59:48.4 for good.

Trust only output from a fresh run	`20261030-07`
	`20261030-16`
	`20261031-12`
		Restart and Run All before trusting a number; close output files in Excel before writing them.

SQL run order (the reason behind several patterns)

```
FROM / JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT
```

Thursday, 29 October 2026

Focus	Dialer file clean-up in pandas, the not-connected leads report in PostgreSQL (Metabase), and tagging Active-but-unreachable clients as automatic traders in Excel.
Outcome	13 entries: 4 errors, 3 wrong results and 6 how-tos. Final report delivered as Not_Connected_Report_v3.xlsx with 205 clients tagged (High 122, Medium 34, Low 49).
Cards	`20261029-01` to `20261029-13`

`20261029-01` Name the dialer output file with the Start Time date range

pandas · **How-to** · dates, f-string, files

PROBLEM

The combined dialer file needed a name showing the first and last `Start Time` dates.

CAUSE

The dates were stored in variables named `min` and `max`, which replaces Python's built-in functions. Saving the output into the input folder would also break the "exactly 2 .xlsx files" check on the next run.

FIX

```
concat_df['Start Time'] = pd.to_datetime(concat_df['Start Time'], format='%Y-%m-%d %H:%M:%S.%f')
min_date = concat_df['Start Time'].min()
max_date = concat_df['Start Time'].max()

if min_date.date() == max_date.date():
    output_name = f"st_daily_{min_date:%Y-%m-%d}.xlsx"
else:
    output_name = f"st_daily_{min_date:%Y-%m-%d}_to_{max_date:%Y-%m-%d}.xlsx"

output_dir = r'C:\Users\Administrator\Documents\Dialer\st_output'
os.makedirs(output_dir, exist_ok=True)
concat_df.to_excel(os.path.join(output_dir, output_name), index=False)
```

WHY IT WORKS

- Inside an f-string, the text after the colon is a date format: `{min_date:%Y-%m-%d}` gives `2026-10-29`.
- `.date()` drops the time, so the `if` compares calendar days only.
- `os.makedirs(..., exist_ok=True)` creates the output folder only if it is missing.
- Windows file names can't contain colons, so use `%Y%m%d_%H%M` if you need times in the name.

> **Lesson:** Never name a variable after a built-in (`min`, `max`, `list`, `sum`). Keep outputs out of input folders.

`20261029-02` Splitting Caller Ids: [1] vs .str[1], and "'DataFrame' object has no attribute 'str'"

pandas · **Error** · strings, duplicate columns, .str

PROBLEM

`uno_copy['Caller Ids'].str.split('|')[1]` gave the wrong result. The next attempt failed with `AttributeError: 'DataFrame' object has no attribute 'str'`.

CAUSE

`[1]` picks row label 1 of the whole column, not item 1 of each list. The AttributeError happened because two columns were named `Caller Ids`, so selecting the name returned a DataFrame, which has no `.str`.

FIX

```
# Find duplicated column names
print(uno_copy.columns[uno_copy.columns.duplicated()])

# Keep the first copy of each name (only if the copies are identical)
uno_copy = uno_copy.loc[:, ~uno_copy.columns.duplicated()]

# '917557160434 | 919810007919' -> '919810007919'
uno_copy['Caller Ids'] = uno_copy['Caller Ids'].str.split('|').str[1].str.strip()
```

WHY IT WORKS

- `.str` applies a method to every value. `.str[1]` takes item 1 from each row's list.
- `columns.duplicated()` is True for each repeat after the first. `~` flips it, and `.loc[:, mask]` keeps the True columns.
- If the duplicate columns hold different data, rename them (`Caller Ids_1`) instead of dropping.
- Rows without `|` become NaN. Add `.fillna(uno_copy['Caller Ids'])` to keep the original value there.

Lesson: If selecting one column name returns a DataFrame, there are duplicate column names.

20261029-03 Extract customer and agent_id from Contact Info with regex

pandas · **How-to** · regex, str.extract, nullable ints

PROBLEM

`Contact Info` holds text like `userId : 30628867 | callFirst : agent | customer : 919810007919 | agent_id : 8943 | agentType : agent_id`. Only `customer` and `agent_id` were needed.

CAUSE

Splitting on `|` depends on field order. A regex per field finds the value wherever it sits.

FIX

```
uno_copy['customer'] = uno_copy['Contact Info'].str.extract(r'customer\s*:\s*(\d+)', expand=False)
uno_copy['agent_id'] = uno_copy['Contact Info'].str.extract(r'\bagent_id\s*:\s*(\d+)', expand=False)

uno_copy['customer'] = uno_copy['customer'].str[-10:]          # drop the 91 prefix
uno_copy['agent_id'] = pd.to_numeric(uno_copy['agent_id'], errors='coerce').astype('Int64')
```

WHY IT WORKS

- `\s*` = any number of spaces, `\d+` = one or more digits, `( )` = the part returned.
- `\b` (word boundary) stops `sub_agent_id` from matching. `agentType : agent_id` doesn't match because no digits follow.
- `expand=False` returns one column. Rows without a match get NaN, not an error.
- `Int64` (capital I) allows blanks; plain `int64` would turn the column into floats like `8943.0`.

Lesson: One regex per field, so the order of fields doesn't matter.

20261029-04 Combine Call Date and a 12-hour Call Time into one datetime

pandas · **How-to** · dates, parsing, 12-hour clock

PROBLEM

`Call Date` (2026-10-28) and `Call Time` (11:58:14 pm) were separate columns.

CAUSE

The time uses a 12-hour clock with am/pm, so it needs `%I` and `%p`, not `%H`.

FIX

```
uno_copy['Call Datetime'] = pd.to_datetime(
    pd.to_datetime(uno_copy['Call Date']).dt.strftime('%Y-%m-%d')
    + ' '
    + uno_copy['Call Time'].astype(str).str.strip().str.upper(),
    format='%Y-%m-%d %I:%M:%S %p'
)
# '2026-10-28' + ' ' + '11:58:14 PM' -> 2026-10-28 23:58:14
```

WHY IT WORKS

- The inner `pd.to_datetime` makes it work whether `Call Date` is text or a datetime.
- `.dt.strftime` turns each date into text, and `+` joins text columns row by row.
- `.str.upper()` turns `pm` into `PM` so it matches `%p`.
- To find bad rows, add `errors='coerce'` and look at rows where the result `.isna()`.

Lesson: %H = 24-hour clock. %I + %p = 12-hour clock with AM/PM.

20261029-05 Leads whose last 5 calls are all one disposition

SQL · **Wrong result** · ROW_NUMBER, HAVING, BOOL_AND

PROBLEM

The original query used `DISTINCT ON` to get only the latest disposition. Filtering for `'0'` found nothing, and the old filter on Call Later / Interested / Prospect would always return 0 rows.

CAUSE

`DISTINCT ON` keeps one row per lead, so it can't look at 5 calls. The stored name is `'Zero'`, not `'0'`. A lead whose last 5 calls are Zero can't also have Interested as its last call.

FIX

```
ranked_calls AS (
    SELECT cl.lead_id, cl.started_at, d.name AS disposition,
           ROW_NUMBER() OVER (PARTITION BY cl.lead_id
                              ORDER BY cl.started_at DESC, cl.id DESC) AS rn
    FROM call_logs cl
    JOIN old_leads ol              ON ol.id = cl.lead_id
    JOIN lead_calls lc             ON lc.call_log_id = cl.id
    JOIN lead_call_dispositions d  ON d.id = lc.disposition_id
    WHERE cl.status NOT IN ('placing', 'call_failed')
),
last_5_zero AS (
    SELECT lead_id, MAX(started_at) AS last_call_at
    FROM ranked_calls
    WHERE rn <= 5
    GROUP BY lead_id
    HAVING COUNT(*) = 5
       AND BOOL_AND(COALESCE(TRIM(disposition) = 'Zero', false))
)
```

WHY IT WORKS

- `ROW_NUMBER` restarts at 1 for each lead (`PARTITION BY`); 1 is the newest call. `cl.id DESC` breaks ties.
- `HAVING` filters groups after `GROUP BY`; `WHERE` filters rows before.
- `COUNT(*) = 5` requires at least 5 calls. `BOOL_AND` is true only if every row passes.
- `BOOL_AND` skips NULLs, so `COALESCE(..., false)` makes a blank disposition break the streak.

Lesson: Rank, cut, group, test. And check the stored value before filtering on it.

20261029-06 Raw verification query showed non-Zero dispositions

SQL · **Wrong result** · window functions, WINDOW, verification

PROBLEM

The raw-data check listed rows with other dispositions, so it looked like the main query was wrong.

CAUSE

The check listed the last 5 calls of every lead, not only qualifying ones. The flags were window results, which can't be filtered in the same SELECT that creates them.

FIX

```
last_5 AS (
    SELECT rc.*,
           COUNT(*) OVER w AS calls_in_last_5,
           BOOL_AND(COALESCE(TRIM(rc.disposition) = 'Zero', false)) OVER w AS all_zero
    FROM ranked_calls rc
    WHERE rc.rn <= 5
    WINDOW w AS (PARTITION BY rc.lead_id)
)
SELECT l.lead_id, l.rn AS call_rank,
       l.started_at AT TIME ZONE 'Asia/Kolkata' AS call_time, l.disposition
FROM last_5 l
WHERE l.calls_in_last_5 = 5 AND l.all_zero
ORDER BY l.lead_id, l.rn;
```

WHY IT WORKS

- Adding `OVER (...)` to an aggregate gives each row its group's total without collapsing rows.
- `WINDOW w AS (...)` names the window once so every `OVER w` reuses it.
- Run order: FROM/JOIN, WHERE, GROUP BY, HAVING, window functions, SELECT, ORDER BY, LIMIT. So filter window results in the next step.
- Check: distinct `lead_id` in this output must equal the main query's row count.

Lesson: Calculate in one CTE, filter in the next. Always compare a summary with its raw rows.

20261029-07 Latest disposition is Zero, plus the distribution of the last 5

SQL · **How-to** · FILTER, STRING_AGG, two-level aggregation

PROBLEM

Needed leads whose latest call is Zero, with counts of each disposition in the last 5 and their order.

CAUSE

Counting per disposition and then per lead needs two levels of `GROUP BY`.

FIX

```
latest_zero AS (
    SELECT lead_id FROM last_5 WHERE rn = 1 AND disposition = 'Zero'
),
disp_counts AS (
    SELECT l.lead_id, l.disposition, COUNT(*) AS cnt
    FROM last_5 l JOIN latest_zero lz ON lz.lead_id = l.lead_id
    GROUP BY l.lead_id, l.disposition
),
per_lead AS (
    SELECT lead_id,
           SUM(cnt) AS calls_in_last_5,
           COALESCE(SUM(cnt) FILTER (WHERE disposition = 'Zero'), 0) AS zero_count,
           STRING_AGG(disposition || ' x' || cnt, ', ' ORDER BY cnt DESC, disposition) AS
    disposition_breakdown
    FROM disp_counts
    GROUP BY lead_id
)
-- sequence: STRING_AGG(disposition, ' -> ' ORDER BY rn)
```

WHY IT WORKS

- `FILTER (WHERE ...)` feeds only matching rows into an aggregate.
- `SUM` over zero rows is NULL, so wrap it in `COALESCE(..., 0)`.
- `STRING_AGG` skips NULLs, so replace NULL dispositions with `'(blank)'` first.
- Check: leads with `zero_count = 5` must equal entry 20261029-05's result.

Lesson: Count at the finest level first, then roll up.

20261029-08 Unreachable list, name normalisation, and last-10 pivot with call times

SQL · **How-to** · regexp_replace, VALUES, pivot

PROBLEM

Last 5 calls must all be one of 8 unreachable dispositions, with the last 10 shown as columns with IST times.

CAUSE

Names like `Did not Connected /Out of Service` break exact matches. `lead_call_dispositions.created_at` is when the option was set up in the CRM, not when a call happened.

FIX

```
target_dispositions AS (
    SELECT regexp_replace(lower(name), '[^a-z0-9]', '', 'g') AS disp_key
    FROM (VALUES ('Ringing/Busy'), ('Switched off'), ('Invalid Number'),
                 ('Did not Connected /Out of Service'), ('Disconnected by customer'),
                 ('Zero'), ('Didn''t connect'), ('No Answer')) AS v(name)
),
last_10 AS (
    SELECT lead_id, rn,
           started_at AT TIME ZONE 'Asia/Kolkata' AS call_time_ist,
           COALESCE(TRIM(disposition), '(blank)') AS disposition,
           regexp_replace(lower(COALESCE(disposition, '')), '[^a-z0-9]', '', 'g')
               IN (SELECT disp_key FROM target_dispositions) AS is_target
    FROM ranked_calls
    WHERE rn <= 10
)
-- pivot, one pair per call number:
-- MAX(l.disposition)   FILTER (WHERE l.rn = 1) AS call_1,
-- MAX(l.call_time_ist) FILTER (WHERE l.rn = 1) AS call_1_time
```

WHY IT WORKS

- `[^a-z0-9]` = any character that isn't a lowercase letter or digit; `'g'` replaces all of them.
- Both sides are normalised, so spacing, capitals and curly apostrophes don't matter.
- `'Didn''t connect'`: two single quotes inside an SQL string = one apostrophe.
- Each lead has one row per `rn`, so `MAX(...) FILTER (WHERE rn = N)` simply pulls that row into a column.

Lesson: Normalise both sides before comparing names. Call times come from `call_logs.started_at`.

20261029-09 column ol.client_id does not exist

SQL · **Error** · Metabase, column names

PROBLEM

Adding `ol.client_id` to the final SELECT failed.

CAUSE

`old_leads` is `SELECT a.*` from saved question 1717, and that question has no column called `client_id`.

FIX

```
-- See the real column names first
SELECT * FROM {{#1717-leads-analysis}} LIMIT 1;

-- Names with capitals or spaces need double quotes
SELECT ol."Client ID" ...

-- The right column turned out to be:
SELECT ol.customer_id ...
```

WHY IT WORKS

- Unquoted names are turned into lowercase, so `Client_ID` needs `"Client_ID"`.
- Questions built in Metabase's GUI with a join may prefix names, e.g. `"Clients__client_id"`.

Lesson: Look at the real column names before guessing one.

20261029-10 syntax error at or near "=" (Position 5128)

SQL · **Error** · JOIN syntax, aliases

PROBLEM

The query failed after adding trading-profile columns.

CAUSE

`left join ol.customer_id = ctp.customer_id` has no table name, no alias and no `ON`.

FIX

```
-- Wrong
left join ol.customer_id = ctp.customer_id

-- Right: JOIN <table> <alias> ON <condition>
LEFT JOIN customer_trading_profiles ctp ON ctp.customer_id = ol.customer_id

-- Give every AT TIME ZONE expression a name
ctp.last_txn_date AT TIME ZONE 'Asia/Kolkata' AS last_txn_date
```

WHY IT WORKS

- `Position: 5128` is the character count from the start of the query; look near it for the quoted word.
- Without an alias, an `AT TIME ZONE` column comes out named `timezone`.

Lesson: A join always reads JOIN <table> <alias> ON <condition>.

20261029-11 Leads repeated after joining customer_status_history

SQL · **Wrong result** · one-to-many join, DISTINCT ON, ARRAY_AGG

PROBLEM

Each lead appeared once per status change after the join.

CAUSE

The status table has many rows per customer. Joining it directly multiplies the lead rows.

FIX

```
csh_dedup AS (
    SELECT DISTINCT ON (customer_id, occurred_at)
           customer_id, occurred_at,
           NULLIF(TRIM(previous_status::text), '')                AS previous_status,
           COALESCE(NULLIF(TRIM(new_status::text), ''), '(blank)') AS new_status
    FROM customer_status_history
    WHERE customer_id IN (SELECT customer_id FROM old_leads)
    ORDER BY customer_id, occurred_at, created_at DESC, id DESC
),
status_summary AS (
    SELECT customer_id,
           (ARRAY_AGG(new_status ORDER BY occurred_at DESC))[1] AS rt_status,
           MAX(occurred_at)                                     AS rt_status_since,
           COUNT(*)                                             AS status_events,
           STRING_AGG(new_status || ' (' ||
                      TO_CHAR(occurred_at AT TIME ZONE 'Asia/Kolkata', 'DD Mon YYYY HH24:MI') || ')',
                      ' -> ' ORDER BY occurred_at)              AS status_journey
    FROM csh_dedup
    GROUP BY customer_id
)
-- final: LEFT JOIN status_summary ss ON ss.customer_id = ol.customer_id
```

WHY IT WORKS

- `DISTINCT ON (a, b)` keeps the first row per (a, b) in `ORDER BY` order; `created_at DESC` makes the newest row win.
- `(ARRAY_AGG(... ORDER BY ... DESC))[1]` = the latest value. PostgreSQL arrays start at 1.
- `::text` stops `TRIM` failing if the status column is an enum.
- Join on `ol.customer_id`, not `ctp.customer_id`, so leads with no trading profile keep their status.

Lesson: Collapse many-rows tables to one row per key before joining. Check COUNT(*) = COUNT(DISTINCT lead_id).

`20261029-12`

TypeError: Can only merge Series or DataFrame objects, a <class 'str'> was passed

pandas · **Error** · merge, latest row per key

PROBLEM

`pd.merge('df', 'nc_led', ...)` failed, and the result kept an extra `ClientID` column.

CAUSE

The quotes made the names text, not DataFrames. The merge also used the full ledger instead of the one-row-per-client balance, which would repeat leads.

FIX

```
nc_led['CreatedAt'] = pd.to_datetime(nc_led['CreatedAt'])

nc_led_balance = (nc_led.sort_values('CreatedAt', ascending=False)
                        .drop_duplicates(subset='ClientID', keep='first')[['ClientID', 'Balance']])

df_bal = pd.merge(df,
                  nc_led_balance.rename(columns={'ClientID': 'zynixx_client_id'}),
                  on='zynixx_client_id', how='left', validate='many_to_one')
```

WHY IT WORKS

- Sort newest first, then `drop_duplicates(keep='first')` = latest row per client (pandas' `DISTINCT ON`).
- Renaming the key and using `on=` leaves one key column. Or add `.drop(columns='ClientID')`.
- `validate='many_to_one'` raises an error if a client appears twice on the right.
- If `Balance` is empty everywhere, the key types differ: align them with `astype('string')`.

Lesson: Variables have no quotes. Merge one row per key, and let `validate=` check it.

`20261029-13`

Tag Active-but-unreachable clients as automatic traders

Excel · **How-to** · IF, COUNTIF, confidence levels

PROBLEM

The manager asked for clients who are Active but don't answer calls, tagged as automatic traders.

CAUSE

Being Active means different things depending on when they traded, so a confidence level was added.

FIX

```
AL  unanswered_call_days
=IF(COUNT(I2,K2,M2,O2,Q2)<5,"",1+(INT(I2)<>INT(K2))+(INT(K2)<>INT(M2))+(INT(M2)<>INT(O2))+(INT(
    O2)<>INT(Q2)))

AM  customer_tag
=IF(AG2="Active","Automatic Trader","")

AN  auto_trader_confidence
=IF(AM2="","",IF(OR(AL2=1,AE2<=Q2),"Low",IF(AE2>I2,"High","Medium")))

I = call_1_time (latest)   Q = call_5_time (streak start)   AE = last_txn_date   AG = rt_status
```

WHY IT WORKS

- `INT()` drops the time from a date-time; calls are sorted newest first, so each date change is a new day.
- Low: all 5 calls on one day, or last trade before the streak. High: traded after the latest missed call. Medium: in between.
- Result: 205 tagged (High 122, Medium 34, Low 49). Missed calls are 71% No Answer; only 1 Invalid Number.
- Blank confidence = not Active, by design (315 in Clients, 19,882 in Leads).

Lesson: Give a tag a confidence level when the evidence is uneven, and say what would confirm it.

Friday, 30 October 2026

Focus	Ledger checks in pandas, Excel formula fixes, and combining 60M+ rows of transaction files from two folders (CSV and XLSX) for the migration.
Outcome	21 entries: 8 errors, 6 wrong results and 7 how-tos. Built transactions_loader.py so the transaction files load without crashing the kernel.
Review	Repeated from day 1: a built-in used as a name (all), duplicate column names (four amount columns), and latest-row-per-key sorted by the wrong column. Focus tomorrow: PG40 Day 1, exercises 1 to 10.
Cards	`20261030-01` to `20261030-21`

20261030-01 Select ledger rows for yesterday

pandas · **How-to** · dates, half-open range, sort_values

PROBLEM

Needed only yesterday's ledger rows. The code also ran `led.sort_values(...)` on its own line.

CAUSE

A date filter needs a start and an end. `sort_values` returns a sorted copy, so without `led = ...` the sort is thrown away.

FIX

```
led = led.sort_values('CreatedAt', ascending=False)      # assign it back

today = pd.Timestamp.today().normalize()      # today 00:00:00
yesterday = today - pd.Timedelta(days=1)

led_yesterday = led[(led['CreatedAt'] >= yesterday) & (led['CreatedAt'] < today)]
```

WHY IT WORKS

- `>=` start and `<` end (a half-open range) covers every second, including 23:59:59.5.
- Each condition in brackets, joined with `&`; plain `and` doesn't work on columns.
- If `led['CreatedAt'].max()` is older than yesterday, 0 rows is the correct answer.
- UTC times are 4 hours behind Dubai: convert with `tz_localize('UTC').tz_convert('Asia/Dubai')` first.

Lesson: Use >= start and < end for date ranges, and always assign sort_values back.

20261030-02 The combined ledger seemed to stop at the 27th

Polars · **Wrong result** · ledger_loader, diagnostics, Excel-saved CSV

PROBLEM

`get_final_ledger()` looked like it had no rows after the 27th, although files for 28 and 29 Oct existed.

CAUSE

The loader parses `2026-10-29T23:59:53.103Z` correctly. The likely cause was viewing an unsorted `head()`, or a file missing from the append folder or saved through Excel.

FIX

```
print(led['CreatedAt'].max())          # step 1: is the data really missing?

summary = lambda lf: lf.select(
    pl.col("CreatedAt").min().alias("first"),
    pl.col("CreatedAt").max().alias("last"),
    pl.col("CreatedAt").null_count().alias("unparsed_dates"),
    pl.len().alias("rows"),
).collect()
print(summary(scan_base_ledger()))
print(summary(scan_append_files()))
```

WHY IT WORKS

- Check the data before changing the code: `max()` answers "is it there?" in one line.
- `unparsed_dates` above 0 means some date text didn't match the format, usually after Excel saved it.
- `trade_id` shown as `2.61029E+17` means the file was opened in Excel; if saved, the IDs are rounded for good.

Lesson: Measure first (min, max, nulls, rows per source), then decide what's broken.

20261030-03 SUMIFS returned 0 for negative balance protection

Excel · **Wrong result** · SUMIFS, $ references, COUNTIF

PROBLEM

`=SUMIFS(...!I:I, ...!E:E, Sheet2!D3, ...!C:C, Sheet2!A4)` gave 0 down the column.

CAUSE

`D3` had no `$`, so it moved to `D4`, `D5`... when copied down. The ledger type may also be spelled differently, or there may be no such entries that day.

FIX

```
=SUMIFS('direct_ledger_report_filtered ('!$I:$I,
        'direct_ledger_report_filtered ('!$E:$E, D$3,
        'direct_ledger_report_filtered ('!$C:$C, $A4)

=COUNTIF('direct_ledger_report_filtered ('!E:E, D3)            exact matches
=COUNTIF('direct_ledger_report_filtered ('!E:E, "*NEGATIVE*")   any spelling
```

WHY IT WORKS

- `D$3` fixes the row (a header); `$A4` fixes the column (a label); `$I:$I` fixes both.
- `SUMIFS` ignores capitals but not underscores or trailing spaces. `*` in a criterion = any characters.
- Easier: put `ledger_type` in the pivot table's Columns area to get every type at once.

Lesson: Anchor headers with D$3 and labels with $A4 before copying a formula.

20261030-04 Extract the number from BR575346: ****

Excel · **How-to** · REGEXEXTRACT, MID, FIND

PROBLEM

Needed `575346` out of IDs like `BR575346: ****`, where the letter prefix varies in length.

CAUSE

The digits start at a different position for one-letter (`S75577`) and two-letter prefixes.

FIX

```
=REGEXEXTRACT(A2, "[0-9]+")                                  Microsoft 365

=MID(A2, MIN(FIND({0,1,2,3,4,5,6,7,8,9}, A2&"0123456789")),     older Excel
     FIND(":", A2) - MIN(FIND({0,1,2,3,4,5,6,7,8,9}, A2&"0123456789")))
```

WHY IT WORKS

- `[0-9]+` = one or more digits in a row; `--` in front turns the text into a number.
- `MIN(FIND({0..9}, ...))` = position of the first digit; adding `"0123456789"` stops `FIND` erroring.
- pandas: `str.extract(r'(\d+)', expand=False)`. PostgreSQL: `substring(col FROM '[0-9]+')`.

Lesson: A regex finds a pattern wherever it sits; position-based formulas break when prefixes vary.

20261030-05 The formula shows as text instead of calculating

Excel · **Error** · cell format, Show Formulas

PROBLEM

`=REGEXEXTRACT(A2, "[0-9]+")` stayed visible as text in the cell.

CAUSE

The cell was formatted as Text, so Excel stored the formula as plain text.

FIX

```
1. Select the cells -> Home -> Number format -> General
2. Re-enter: F2, then Enter (changing the format alone doesn't recalculate)
3. Many cells: Ctrl+H, replace = with =, Replace All
```

WHY IT WORKS

- Other causes: Show Formulas is on (Formulas -> Show Formulas), or a space or apostrophe before the `=`.
- If it then shows `#NAME?`, the Excel version has no `REGEXEXTRACT`; use the `MID` version.

Lesson: A formula that won't calculate is usually a Text-formatted cell.

20261030-06 UNIQUE(FILTER(..., OR(...))) doesn't filter

Excel · **Wrong result** · FILTER, OR logic, arrays

PROBLEM

`=UNIQUE(FILTER(dgrdReport!G:G, OR(D:D="Buy", D:D="Sell")))` gave an error or wrong rows.

CAUSE

`OR()` returns one TRUE/FALSE for the whole column; `FILTER` needs one per row.

FIX

```
=UNIQUE(FILTER(dgrdReport!G:G,
               (dgrdReport!D:D="Buy") + (dgrdReport!D:D="Sell"),
               "No matches"))

=UNIQUE(FILTER(dgrdReport!G:G, ISNUMBER(MATCH(dgrdReport!D:D, {"Buy","Sell"}, 0)), "No matches"))
```

WHY IT WORKS

- `+` turns TRUE/FALSE into 1/0 row by row; `FILTER` keeps non-zero rows. `*` means AND.
- `"No matches"` replaces the `#CALC!` error when nothing matches.
- Whole columns like `G:G` are slow; fixed ranges like `G2:G50000` are faster.

Lesson: Inside FILTER: + for OR, * for AND. Never OR() or AND().

20261030-07 KeyError: 'ClientID' in groupby

pandas · **Error** · variable names, columns

PROBLEM

`df.groupby('ClientID', as_index=False)['CreatedAt'].max()` raised `KeyError: 'ClientID'`.

CAUSE

`df` had just been replaced by `pd.read_excel(...)`, so it held the traded-IDs sheet, not the ledger.

FIX

```
print(df.columns.tolist())                    # what does df actually hold?

last_dates = led.groupby('ClientID', as_index=False)['CreatedAt'].max()

df.columns = df.columns.str.strip()           # if names have hidden spaces
df = df.reset_index()                         # if ClientID became the index
```

WHY IT WORKS

- A `KeyError` on a column means that exact name isn't in `df.columns`: wrong table, spaces, capitals, or index.
- `pd.read_excel(path, sheet_name='Sheet1')` reads one tab; `pd.ExcelFile(path).sheet_names` lists them.

Lesson: Give every table its own name (led, traded_ids); reusing df hides which table you have.

20261030-08 Fill Zynixx Balance and Last Trade with .map()

pandas · **How-to** · map, lookup, set_index

PROBLEM

The traded-IDs sheet had empty `Zynixx Balance` and `Zynixx Last T` columns to fill from two small tables.

CAUSE

`merge` would add new key and value columns. `.map()` fills an existing column and keeps the row order.

FIX

```
bal_map  = balance.drop_duplicates('ClientID').set_index('ClientID')['Balance']
date_map = last_dates.drop_duplicates('ClientID').set_index('ClientID')['CreatedAt']

key = df['Zynixx Id'].astype(str).str.strip()
df['Zynixx Balance'] = key.map(bal_map)
df['Zynixx Last T']  = key.map(date_map)
```

WHY IT WORKS

- `set_index(key)[col]` turns a table into a lookup: labels are IDs, values are the answers.
- IDs not found (like `No New Id`) become NaN; `df` keeps its 212 rows.
- Duplicate IDs in the lookup make `.map()` fail; `drop_duplicates` prevents it.

Lesson: To fill a column from another table by key, use .map() on a one-row-per-key lookup.

20261030-09 Latest balance taken from a random row

pandas · **Wrong result** · drop_duplicates, latest row per key

PROBLEM

`led_req.sort_values('ClientID').drop_duplicates(subset='ClientID', keep='first')` gave balances that weren't the latest.

CAUSE

Sorting by `ClientID` groups each client's rows but leaves them in no date order, so `keep='first'` picks any row.

FIX

```
balance = (led_req.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first')
                  .reset_index(drop=True)[['ClientID', 'Balance']])
```

WHY IT WORKS

- "Latest row per key" = sort by the DATE, newest first, then keep the first row per key.
- Same idea as `DISTINCT ON (key) ... ORDER BY key, date DESC` in PostgreSQL.

Lesson: keep='first' is only as good as the sort before it: sort by the column that defines 'first'.

20261030-10 KeyError: Index([Ellipsis], dtype='object')

pandas · **Error** · placeholder code

PROBLEM

Running a comparison block failed inside `drop_duplicates`.

CAUSE

The block included `drop_duplicates(...)` with a literal `...`, written as a placeholder. Python treats `...` as a real value (`Ellipsis`) and looked for a column with that name.

FIX

```
# Run only the real line, never the illustrative one:
balance = (led_req.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first'))
```

WHY IT WORKS

- `...` is valid Python, so it doesn't fail until the function uses it.
- When a snippet shows a "before" and "after", copy only the "after".

Lesson: Read a code block before running it; skip lines marked as examples of what not to do.

20261030-11 Add a result sheet to the original workbook

pandas · **How-to** · ExcelWriter, openpyxl

PROBLEM

Needed the finished `df` saved as a sheet called `result` in the original Excel file.

CAUSE

`df.to_excel(path)` alone overwrites the whole workbook.

FIX

```
with pd.ExcelWriter(path, engine='openpyxl', mode='a', if_sheet_exists='replace') as writer:
    df.to_excel(writer, sheet_name='result', index=False)
```

WHY IT WORKS

- `mode='a'` adds to the file and keeps the other sheets.
- `if_sheet_exists='replace'` overwrites `result` on reruns instead of creating `result1`.
- Close the file in Excel first (`PermissionError`), and note `openpyxl` can drop pivot tables and slicers.

Lesson: Append a sheet with ExcelWriter(mode='a'); write a new file when the workbook has pivots.

20261030-12 Combining every file in a folder

pandas · **Error** · os.listdir, concat, built-in names

PROBLEM

The loop started with `all = []`, called `pd.read_csv()` with no path, and created an empty table on every pass.

CAUSE

`all` is a built-in function; `read_csv` needs the full path; `.xlsx` needs `read_excel`.

FIX

```
frames = []
for file in os.listdir(folder):
    full_path = os.path.join(folder, file)
    if file.startswith('~$'):
        continue
    if file.lower().endswith('.csv'):
        part = pd.read_csv(full_path, dtype=text_cols)
    elif file.lower().endswith('.xlsx'):
        part = pd.read_excel(full_path)
    else:
        continue
    part['source_file'] = file
    frames.append(part)

combined = pd.concat(frames, ignore_index=True)
```

WHY IT WORKS

- Collect pieces in a list and `concat` once; it is much faster than growing a table file by file.
- `os.listdir` returns names only, so `os.path.join` builds the path.
- `~$` files are Excel lock files and can't be read.

Lesson: Same lesson as min and max yesterday: never use a built-in name (all, list, sum) for a variable.

20261030-13

DtypeWarning: Columns (21: expiryDate) have mixed types

pandas · **Error** · dtype, read_csv, mixed types

PROBLEM

`read_csv` warned about `expiryDate`, `comment` and `scriptParent` in several files.

CAUSE

Big files are read in chunks and each chunk's type is guessed separately. Blank chunks look like numbers and others hold text, so the column ends up mixed.

FIX

```
text_cols = {'expiryDate': str, 'comment': str, 'scriptParent': str}
part = pd.read_csv(full_path, dtype=text_cols)

combined['expiryDate'] = pd.to_datetime(combined['expiryDate'], errors='coerce')
combined['expiryDate'].map(type).value_counts()      # check what a column holds
```

WHY IT WORKS

- A mixed column breaks sorting (`'<' not supported between 'str' and 'float'`) and merges.
- `low_memory=False` only hides the warning; the values are still mixed.

Lesson: Read mostly-empty or mixed columns as text, then convert them once on purpose.

20261030-14 MemoryError: Unable to allocate 487 MiB ... shape (1, 63807297)

pandas · **Error** · memory, usecols, concat

PROBLEM

`pd.concat(frames)` failed after reading every transaction file.

CAUSE

63,807,297 rows × 8 bytes = 487 MiB per number column. With ~30 columns the table needs 15+ GB, and `concat` holds the pieces and the copy at once.

FIX

```
part = pd.read_csv(full_path, usecols=keep_cols, dtype=text_cols)   # only needed columns
part = part[part['clientId'].isin(ids)]                              # only needed rows
frames.append(part)
...
combined = pd.concat(frames, ignore_index=True)
del frames                                                           # free the pieces
```

WHY IT WORKS

- Estimate first: rows × columns × 8 bytes for numbers; text costs more.
- Cutting columns while reading saves the most; filtering each file saves the rest.
- For this size, Polars (lazy, parallel) is the better tool.

Lesson: Size the data before loading it; read only the columns and rows you need.

20261030-15 'LazyFrame' object has no attribute 'to_pandas'

Polars · **Error** · LazyFrame, collect

PROBLEM

`lf = lf.to_pandas()` failed.

CAUSE

A LazyFrame is a plan of what to read, not data. Only a collected DataFrame has `.to_pandas()`.

FIX

```
lf.collect_schema().names()     # column names, no data read
lf.head(5).collect()            # first rows only
lf.select(pl.len()).collect()   # row count

result_pd = lf.select(cols).filter(cond).collect().to_pandas()
```

WHY IT WORKS

- `.collect()` runs the plan; `.to_pandas()` then converts the result.
- Don't reuse `lf` for a pandas table; keep `lf` for the plan and new names for data.

Lesson: Lazy = plan. collect() = load. Cut the plan down before collecting.

20261030-16 Kernel crashed on lf.collect()

Polars · **Error** · memory, streaming, sink

PROBLEM

`result = lf.collect()` then `result.to_pandas()` crashed the Jupyter kernel; every variable was lost.

CAUSE

It loaded all 63.8M+ rows and every column, then tried to make a second full copy for pandas.

FIX

```
needed = (lf.select(['clientId', 'symbol', 'time'])
            .filter(pl.col('clientId').is_in(list(ids)))
            .collect(engine='streaming'))                  # older Polars: streaming=True
print(needed.shape, round(needed.estimated_size('gb'), 2), 'GB')

lf.sink_csv(r"C:\...\transactions_combined.csv")       # everything, to disk, in batches
```

WHY IT WORKS

- `engine='streaming'` processes in batches; `group_by(...).agg(...)` before collect returns only totals.
- `sink_csv` / `sink_parquet` write every row without holding it in memory, which suits `\copy` into PostgreSQL.
- Check `estimated_size('gb')` before `.to_pandas()`; converting needs that much again.

Lesson: Never collect a whole big plan. Select, filter, or sink to disk.

20261030-17 Append the ark folder (CSV and XLSX) to the transactions

Polars · **How-to** · read_excel, diagonal_relaxed, source_file

PROBLEM

The ark append folder had both `.csv` and `.xlsx` files to add to the transaction plan.

CAUSE

Polars scans CSVs lazily but must read Excel files fully; both then have to join one plan.

FIX

```
def read_xlsx_file(f):
    return (pl.read_excel(f, infer_schema_length=10_000)     # pip install fastexcel
              .lazy()
              .with_columns(pl.lit(f.name).alias('source_file')))

parts = [scan_csv_file(f) for f in trans_csv + ark_csv] + [read_xlsx_file(f) for f in ark_xlsx]
lf = pl.concat(parts, how='diagonal_relaxed')
```

WHY IT WORKS

- `.lazy()` turns a loaded Excel table into a LazyFrame; each `.xlsx` is at most ~1M rows.
- `diagonal_relaxed` matches columns by name and widens differing types (a date vs text becomes text).
- `pl.lit(f.name)` tags each row with its file, so you can count rows per source.

Lesson: Tag every row with its source file whenever you combine files.

20261030-18 transactions_loader.py: a module that can't crash the kernel

Polars · **How-to** · module, select before collect

PROBLEM

`get_final_ledger(as_pandas=True)` never crashed, but loading the transactions did.

CAUSE

The ledger loader keeps 6 columns before collecting, so Polars reads only those. The transactions code collected 30+ columns.

FIX

```
from transactions_loader import list_columns, check_columns, count_rows, get_transactions

check_columns()                                  # files whose columns differ
count_rows(by_file=True)                         # rows per file, nothing loaded
df = get_transactions(columns=['ticketId', 'time', 'accountId', 'amount'],
                      where=pl.col('accountId').is_in(list(ids)),
                      as_pandas=True)
```

WHY IT WORKS

- `columns` is required, so the full table can't be loaded by accident.
- Results over `MAX_PANDAS_GB` (4 GB) raise a clear error instead of crashing.
- `export_transactions(path)` streams everything to `.parquet` or `.csv` for the migration.

Lesson: Build the safe habit into the tool: make the big, dangerous option impossible by default.

20261030-19 Aligning ark columns gave four amount columns

pandas · **Wrong result** · rename, normalised names, clash check

PROBLEM

After renaming ark columns to the main names, the result had `amount` four times: `1.00, 37372.0, 1.00, 37372.0`.

CAUSE

Two ark columns normalised to `amount`, and selecting `[list(rename.values())]` asked for `amount` twice, so each copy appeared twice. The code had no clash check.

FIX

```
def norm(name):
    return re.sub(r'[^a-z0-9]', '', str(name).lower())      # 'Ticket Id' -> 'ticketid'

targets = pd.Series(rename)
clashes = targets[targets.duplicated(keep=False)]
if not clashes.empty:
    raise ValueError("Same target name:\n" + clashes.to_string())

ark_clean = df[list(rename)].rename(columns=rename)          # select originals, then rename
```

WHY IT WORKS

- Lowercase alone fails: `Ticket Id` -> `ticket id`, not `ticketId`. Removing spaces and symbols matches them.
- Names that differ in words (`Account` -> `accountId`, `Open Com` -> `openCommission`) are mapped by hand.
- Each ark file has different headers (31 vs 14 columns), so align each file on its own.

Lesson: After any rename, check that no two columns share a name.

20261030-20 Match the main file's columns and types exactly (minus id)

pandas · **How-to** · reindex, Int64, dtypes

PROBLEM

The cleaned ark table had 28 of the 30 target columns, in a different order, with `ticketId` as `41672978.0`.

CAUSE

Missing columns, different order, and blanks forcing whole numbers into decimals.

FIX

```
target_cols = [c for c in major.columns if c != 'id']
major_final = major.drop(columns='id')
ark_final = ark_clean.reindex(columns=target_cols)       # adds fileReferenceId, location as blank
assert list(ark_final.columns) == list(major_final.columns)

for df_ in (major_final, ark_final):
    for c in ['ticketId', 'openPosition']:
        df_[c] = pd.to_numeric(df_[c], errors='coerce').astype('Int64')
```

WHY IT WORKS

- `reindex(columns=...)` sets the order and adds missing columns as blanks in one step.
- `Int64` (capital I) allows blanks and keeps whole numbers.
- Compare types side by side: `pd.DataFrame({'major': major_final.dtypes, 'ark': ark_final.dtypes})`.

Lesson: Same names is not enough: make the types match before appending.

20261030-21 Times show as 59:48.4 (date and hour lost)

pandas · **Wrong result** · Excel, CSV, data loss

PROBLEM

`ark_final['time']` held values like `59:48.4` and `openDate` like `28:14.3`.

CAUSE

The CSV was opened in Excel and saved. Excel displays times with fractions as minutes:seconds and saves the display, so the date and hour are gone from the file.

FIX

```
short = ark_clean['time'].astype(str).str.fullmatch(r'\d{1,2}:\d{2}(\.\d+)?')
ark_clean.loc[short, 'source_file'].value_counts()            # which files are damaged
print(ark_clean.duplicated(subset='ticketId').sum())          # same rows in csv AND xlsx?

t = ark_final['time'].astype(str).str.strip()
ark_final['time'] = pd.to_datetime(t.where(~t.str.fullmatch(r'\d{1,2}:\d{2}(\.\d+)?')),
                                   format='ISO8601', errors='coerce')     # blank, don't guess
```

WHY IT WORKS

- The same rows appeared with full times in another ark file, so a clean copy likely exists.
- If the same data is in a `.csv` and an `.xlsx`, every row is loaded twice; removing the damaged file fixes both.
- Never save a CSV from Excel; look at it with `pd.read_csv(path, nrows=20)` instead.

Lesson: Keep original downloads untouched. Excel rewrites what it saves.

Saturday, 31 October 2026

Focus	Rebuilding the Stringee + Uno dialer report as a fast, tested script; making it multi-day; the Uno API token; client trading days.
Outcome	12 entries: 3 errors, 3 wrong results and 6 how-tos. dialer_report.py matched the old notebook cell for cell on Stringee data, fixed three old bugs, and runs a test day in 2.1 s instead of 7.9 s.
Review	Repeated: debugging code before checking inputs (the report read an old folder), and trusting output from an older run (the trading-days file). Focus tomorrow: PG40 Day 1, exercises 1 to 5.
Cards	`20261031-01` to `20261031-12`

20261031-01 FILTER rejected: 'BILL' in single quotes

Excel · **Error** · quotes, FILTER, UNIQUE

PROBLEM

`=unique(FILTER('31 oct 2026'!C:C,'31 oct 2026'!E:E='BILL'))` wouldn't work.

CAUSE

Excel uses single quotes only around sheet names. Text values need double quotes.

FIX

```
=UNIQUE(FILTER('31 oct 2026'!C:C, '31 oct 2026'!E:E="BILL", "No matches"))
```

WHY IT WORKS

- `'31 oct 2026'!` = a sheet name with spaces; `"BILL"` = text.
- `"No matches"` replaces the `#CALC!` error when nothing matches.
- The match ignores capitals but not trailing spaces.

> **Lesson:** Single quotes for sheet names, double quotes for text.

20261031-02 Organised, faster Stringee CDR combine script

pandas · **How-to** · pathlib, calamine, xlsxwriter, # %% cells

PROBLEM

The daily script used `os.chdir`, unused imports, a `> 2` file check and slow Excel engines.

CAUSE

`> 2` let 0 or 1 file through and then failed on `xlsx_files[1]`; Excel lock files counted as inputs.

FIX

```
files = sorted(f for f in INPUT_DIR.glob("*.xlsx") if not f.name.startswith("~$"))
if len(files) != 2:
    raise ValueError(f"Expected exactly 2 .xlsx files, found {len(files)}: {[f.name for f in files]}")

def read_xlsx(path):
    try:
        return pd.read_excel(path, engine="calamine")     # pip install python-calamine
    except ImportError:
        return pd.read_excel(path)
```

WHY IT WORKS

- `Path` objects carry the full folder, so `os.chdir` isn't needed.
- `calamine` reads and `xlsxwriter` writes much faster; the helper falls back if missing.
- `format="ISO8601"` reads times with or without fractions of a second.
- `# %%` cells in a `.py` file convert to `.ipynb` with VS Code or `jupytext --to notebook`.

Lesson: Make a file-count check list the files it found; it turns a crash into an answer.

20261031-03 Read IDs as text; convert dates and durations after

pandas · **How-to** · dtype, to_timedelta, [h]:mm:ss

PROBLEM

Stringee files mix real date-times, `00:05:40` durations and 12-digit phone numbers.

CAUSE

A blank cell turns a phone column into floats (`9.19e+11`), and Excel durations arrive as time-of-day values.

FIX

```
df = pd.read_excel(path, dtype={"ID": str, "Customer number": str, "Hotline": str})
for c in ["Queue duration", "Answer duration", "Hold duration"]:
    df[c] = pd.to_timedelta(df[c].astype(str).str.strip(), errors="coerce")

# writing back: numbers as day fractions + a duration format
out[c] = out[c].dt.total_seconds() / 86_400
writer.sheets["CDR"].set_column(i, i, 14, writer.book.add_format({"num_format": "[h]:mm:ss"}))
```

WHY IT WORKS

- Only IDs need `dtype` at read time; afterwards the float damage is done.
- pandas writes timedeltas as `0.003935`; with `[h]:mm:ss` Excel shows `0:05:40` and `SUM` still works.
- Writing numbers first matters: pandas' own timedelta cells carry a `0` format that overrides the column's.

Lesson: Fix types at the boundary: text IDs on the way in, display formats on the way out.

20261031-04 Old dialer notebook: Uno start times lost and durations zeroed

pandas · **Wrong result** · regression test, Uno, timedelta

PROBLEM

In the old notebook every Uno call landed in the `20+` slot, with no gaps, Total_Duration 00:00:00 and Login Hours 01:00:00.

CAUSE

The date was glued to Uno's full date-time (`'2026-10-30 2026-10-30 09:12:00'`, which can't parse), and a later step set anything not stored as text to `Timedelta(0)`.

FIX

```
uno_start = pd.to_datetime(
    uno["Call Date"].dt.strftime("%Y-%m-%d") + " " + uno["Call Time"].astype(str).str.strip().str.
    upper(),
    format="%Y-%m-%d %I:%M:%S %p", errors="coerce")
uno_calls["Total Call Duration"] = to_td(uno["Call Duration"])     # handles text and timedeltas
```

WHY IT WORKS

- Found by running a faithful copy of the old notebook beside the new script on the same test days.
- Stringee results matched cell for cell; only Uno rows differed, which pointed straight at the bugs.
- A third bug: `int(mins * 60)` cut 1 second from about 2% of hourly talk times (100 s became 99 s).

> **Lesson:** Before replacing a report, run old and new on the same data and explain every difference.

20261031-05 Call gaps with groupby().shift() instead of a loop

pandas · **How-to** · groupby, shift, vectorise

PROBLEM

The gap calculation looped row by row with `.loc`, the slowest part of the notebook.

CAUSE

Each call needs the previous call's start and duration for the same CRE and day.

FIX

```
dialers = dialers.sort_values(["Date", "CRM ID", "Call Start Time"], kind="mergesort").reset_index(
    drop=True)
g = dialers.groupby(["Date", "CRM ID"], sort=False)
prev_start = g["Call Start Time"].shift()
prev_end = prev_start + g["Total Call Duration"].shift()
gap = (dialers["Call Start Time"] - prev_end).clip(lower=pd.Timedelta(0))
```

WHY IT WORKS

- `shift()` moves values down one row inside each group, so the first call of a day gets nothing.
- Sorting first is essential; `mergesort` keeps ties in a fixed order.
- Summary sums use pre-computed columns (`talk_s.where(conn, 0)`) instead of per-group lambdas.
- Test day of 8,162 calls: 7.9 s before, 2.1 s after (0.6 s without saving raw copies).

> **Lesson:** "Previous row in the same group" is a groupby shift, not a loop.

20261031-06 Grouper for 'Date' not 1-dimensional

pandas · **Error** · pivot_table, keys

PROBLEM

After adding `"Date"` to the hourly pivot keys, the Calls pivot failed.

CAUSE

That pivot counted `values="Date"`, so `Date` was both a group key and the value counted.

FIX

```
keys = ["Date", "CRM ID", "Employee code", "Full Name", "Pool", "TL"]
calls = dialers.pivot_table(index=keys, columns="Interval", values="Call Status",
                            aggfunc="count", fill_value=0)
```

WHY IT WORKS

- Count any column that is never blank; `Call Status` always has a value.
- The same report then merges Login Hours on `["CRM ID", "Date"]` so days don't repeat rows.

> **Lesson:** A column can be a key or a value in one pivot, not both.

20261031-07 Multi-day reports with checks that fail loudly

pandas · **How-to** · concat, drop_duplicates, consistency check

PROBLEM

The report needed to cover any number of days from several exports, some overlapping.

CAUSE

The old version accepted exactly 2 Stringee files and 1 Uno file and assumed a single day.

FIX

```
uno = pd.concat([pd.read_csv(f, dtype=uno_text) for f in uno_files], ignore_index=True)
uno = uno.drop_duplicates(subset="Call Id")

check = hourly[on + ["Total Calls"]].merge(summary[on + ["Total_Dialed_Calls"]], on=on, how="left")
print("Hourly totals match the summary:", (check["Total Calls"] == check["Total_Dialed_Calls"]).all())
print("Calls per day:", dialers.groupby(dialers["Date"].dt.strftime("%Y-%m-%d")).size().to_dict())
```

WHY IT WORKS

- 3-day test: each day matched a single-day run; 50 overlapping Uno rows removed.
- Two reports from the same calls must agree; a mismatch means a join or filter went wrong.
- File names show the range: `Summary_2026-10-28_to_2026-10-30.csv`.

Lesson: Print counts per day and cross-check totals before anyone opens the file.

20261031-08 Report showed only the latest date: wrong input folder

pandas · **Wrong result** · inputs, folders, debugging order

PROBLEM

The summary only had 28 Oct, although an export from 1 Oct existed.

CAUSE

`STRINGEE_DIR` pointed to `Documents\Dialer\st_daily`, holding only the 28th. The full export was in `Videos\Office\Dialer\CDR\Stringee`.

FIX

```
for f, p in zip(st_files, st_parts):
    t = pd.to_datetime(p["Start time"], format="%m/%d/%Y %I:%M:%S %p", errors="coerce")
    print(f"{f.parent.name}/{f.name}: {len(p):,} rows | {t.min()} -> {t.max()}")
```

WHY IT WORKS

- 11 rounds were spent on date parsing and header theories; tests ruled both out.
- The file list printed on every run (`20261029_052117.xlsx` ...) held the clue from the start.
- Printing the folder name beside each file shows a wrong folder at a glance.

Lesson: Check which files are read, from which folder, and their date ranges before touching the code.

20261031-09 Parse 10/28/2026 09:34:25 PM safely

pandas · **How-to** · to_datetime, format, month-first

PROBLEM

Stringee shows start times as `10/28/2026 09:34:25 PM`.

CAUSE

A guessing parser can swap day and month: in testing it read `11/03/2026` as 11 March.

FIX

```
st_raw["Start time"] = pd.to_datetime(st_raw["Start time"], format="%m/%d/%Y %I:%M:%S %p",
    errors="coerce")
unread = st_raw["Start time"].isna().sum()
if unread:
    raise ValueError(f"{unread} Stringee start times could not be read")
```

WHY IT WORKS

- `%m/%d/%Y` = month first; `%I` + `%p` = 12-hour clock with AM/PM.
- Real Excel dates that only display this way pass through unchanged.
- `errors="coerce"` + the `unread` check stops the run instead of dropping rows silently.

Lesson: Give date formats explicitly and make failures stop the run.

20261031-10 Use the newest team list in a folder

pandas · **How-to** · pathlib, st_mtime

PROBLEM

The team list file name changes with every download.

CAUSE

A hard-coded file name breaks as soon as a new export arrives.

FIX

```
team_files = sorted(f for f in TEAM_DIR.iterdir()
                    if f.suffix.lower() in (".csv", ".xlsx") and not f.name.startswith("~$"))
team_file = max(team_files, key=lambda f: f.stat().st_mtime)
ref = (pd.read_csv(team_file, dtype={"Dialer Name": str}) if team_file.suffix.lower() == ".csv"
       else read_xlsx(team_file, dtype={"Dialer Name": str}))
```

WHY IT WORKS

- `st_mtime` = last-modified time; `max` picks the newest.
- Only one list is used: combining old and new lists would duplicate CRM IDs.
- Same day: `EXCLUDE_STRINGEE_END_CODES = []` keeps CAN_NOT_MAKE_CALL as dialed, not connected.

Lesson: Point settings at folders, then pick the file by rule and print which one was used.

20261031-11 KeyError on os.environ: token used as the variable name

Python · **Error** · environment variables, setx, secrets

PROBLEM

`TOKEN = os.environ["<the token>"]` raised `KeyError`.

CAUSE

`os.environ[...]` looks a variable up by NAME. The token's value was passed, and no variable has that name.

FIX

```
# once, in a terminal, then fully restart VS Code:
#   setx UNO_API_TOKEN "your-token"
TOKEN = os.environ["UNO_API_TOKEN"]
print("UNO_API_TOKEN" in os.environ)      # True
```

WHY IT WORKS

- Programs read environment variables only when they start, so restart the editor after `setx`.
- A token pasted into chat is exposed: regenerate it and run `setx` again.
- The API endpoint is still to come from the Uno panel; Uno looks like DeepCall (unconfirmed).

Lesson: Secrets live in environment variables, read by name; never in notebooks or chats.

20261031-12 Over 31 trading days in a month

pandas · **Wrong result** · nunique, normalize, assert

PROBLEM

The client report showed 155, 229 and 446 "days" in single months.

CAUSE

It still counted BILL rows. The file came from the old pivot: no `Total Trading Days` column existed.

FIX

```
bill_led["Trade Date"] = bill_led["CreatedAt"].dt.normalize()
pivot = bill_led.pivot_table(index="ClientID", columns="Month", values="Trade Date",
                             aggfunc="nunique", fill_value=0)
assert pivot.max().max() <= 31, "Still counting bills, not days"
```

WHY IT WORKS

- `.dt.normalize()` drops the time; `nunique` counts distinct dates.
- The `assert` stops an impossible result before it reaches Excel.
- Close the Excel file before exporting, or the old version stays on disk.

Lesson: Add a check for the impossible (more than 31 days) so wrong logic can't ship quietly.

Monday, 2 November 2026

Focus	Lead sources for depositors, October traders by manager and agent, a per-client report of Sales calls after conversion, and IDs sharing an IP in Metabase. Also built the 115-exercise mastery workbook.
Outcome	10 entries: 8 how-tos and 2 wrong results. Every join that could repeat rows was guarded before it could cause a problem.
Review	Most extra rounds came from unclear output columns (the 30-day request took four rounds). Focus tomorrow: mastery workbook exercises 1 to 15, then progress().
Cards	`20261102-01` to `20261102-10`

`20261102-01`

Lead source for depositing customers

SQL · **How-to** · DISTINCT ON, self-join, COALESCE

PROBLEM

Needed Lead Source 1 and 2 for every customer with `total_deposit_count > 0`.

CAUSE

A customer can have several leads, so joining `leads` directly repeats depositors. The source has a parent source in the same `sources` table.

FIX

```
WITH lead_source AS (
    SELECT DISTINCT ON (l.customer_id) l.customer_id, l.id AS lead_id,
           COALESCE(s2.name, 'Unknown') AS lead_source_1, s1.name AS lead_source_2
    FROM leads l
    LEFT JOIN sources s1 ON l.source_two_id = s1.id
    LEFT JOIN sources s2 ON s1.parent_id = s2.id
    WHERE l.customer_id IS NOT NULL AND l.deleted_at IS NULL
    ORDER BY l.customer_id, l.date_of_lead, l.created_at, l.id
)
SELECT ctp.*, ls.lead_source_1, ls.lead_source_2
FROM customer_trading_profiles ctp
LEFT JOIN lead_source ls ON ls.customer_id = ctp.customer_id
WHERE ctp.total_deposit_count > 0;
```

WHY IT WORKS

- `sources s1` and `sources s2` are the same table under two aliases: the source and its parent.
- `DISTINCT ON` + `ORDER BY date_of_lead` keeps the first lead; `DESC` would credit the latest.
- `l.deleted_at` carries its alias, so it can't become ambiguous when more tables join.

Lesson: Cut a many-rows table to one row per key in a CTE before joining it.

20261102-02 Flag which customers traded in October

pandas · **How-to** · isin, np.where, Period

PROBLEM

Needed a Traded / Not Traded column on the filtered customer list for October.

CAUSE

The October BILL rows hold the traders' IDs; each customer just needs checking against that set.

FIX

```
led_req = led[led["ClientID"].isin(cust_req)].copy()
led_req = led_req[(led_req["LedgerType"] == "BILL") &
                  (led_req["CreatedAt"].dt.to_period("M") == pd.Period("2026-10", freq="M"))]

traded_ids = set(led_req["ClientID"].astype(str).str.strip())
customer_filtered = customer_filtered.copy()
customer_filtered["Oct Trading"] = np.where(
    customer_filtered["Client ID"].astype(str).str.strip().isin(traded_ids), "Traded", "Not Traded")
```

WHY IT WORKS

- `isin` gives True/False per customer; `np.where` turns it into labels.
- Strip and cast both sides, so spaces or number-vs-text IDs can't hide a match.
- `.str.lower()` before `isin(['demo', 'suspended'])` also catches `Demo`.

Lesson: A flag column is isin + np.where; clean both sides of the match first.

20261102-03 Total and traded clients per Manager and agent

pandas · **How-to** · groupby, nunique, where, subtotals

PROBLEM

Needed, per Manager and Assigned To, the number of clients and how many traded in October.

CAUSE

A conditional distinct count needs only the traders' IDs in one column.

FIX

```
cf = customer_filtered.copy()
cf["Assigned To"] = cf["Assigned To"].fillna("Unassigned")
cf["Traded_ID"] = cf["Client ID"].where(cf["Oct Trading"].eq("Traded"))

pivot = (cf.groupby(["Manager", "Assigned To"])
           .agg(Total_Clients=("Client ID", "nunique"), Traded_in_Oct=("Traded_ID", "nunique"))
           .reset_index())
pivot["Traded %"] = (pivot["Traded_in_Oct"] / pivot["Total_Clients"] * 100).round(1)
```

WHY IT WORKS

- `.where(cond)` blanks non-traders; `nunique` ignores blanks, so it counts only traders.
- `fillna('Unassigned')` stops groupby dropping clients with no agent.
- Subtotals: group by Manager alone, label `Manager total`, concat, sort with a helper `_order` column.

Lesson: Conditional distinct count = .where() then nunique, no lambda needed.

20261102-04 Calls after conversion would repeat per lead

SQL · **Wrong result** · fan-out, MIN, FILTER, STRING_AGG

PROBLEM

The first calls query joined `call_logs`, `customers`, `customer_trading_profiles` and `leads` together.

CAUSE

A customer with two leads gets every call twice. Grouping only by `zynixx_client_id` would also merge customers with no trading profile.

FIX

```
WITH conv AS (
    SELECT customer_id, MIN(converted_at) AS converted_at
    FROM leads WHERE converted_at IS NOT NULL GROUP BY customer_id
)
SELECT customer_id, zynixx_client_id,
       COUNT(*)                                                    AS calls_after_conversion,
       COUNT(*) FILTER (WHERE since_conversion <= INTERVAL '1 day') AS within_1_day,
       COUNT(answered_at)                                          AS answered_after_conversion,
       STRING_AGG(TO_CHAR(started_at AT TIME ZONE 'Asia/Kolkata', 'DD Mon YY HH24:MI'), ', ' ORDER BY
    started_at) AS call_times
FROM calls_after
GROUP BY customer_id, zynixx_client_id;
```

WHY IT WORKS

- `MIN(converted_at)` per customer leaves one conversion date, so calls can't multiply.
- `COUNT(*)` counts rows; `COUNT(answered_at)` counts only non-blank values, so answered calls.
- Compare stored times (`started_at > converted_at`); convert time zones only for display.

Lesson: Before joining a table, ask: how many rows per key does it have?

20261102-05 Start from the pasted ID list so no ID disappears

SQL · **How-to** · UNNEST, ARRAY, LEFT JOIN, check_status

PROBLEM

With `WHERE zynixx_client_id IN (...)`, IDs with no calls, or that didn't exist, vanished from the result.

CAUSE

The query started from calls; an ID without calls never made it into a row.

FIX

```
WITH id_list AS (
    SELECT DISTINCT UNNEST(ARRAY['S221954', 'P227319', 'A226436']) AS zynixx_client_id
)
SELECT i.zynixx_client_id,
       CASE WHEN i.customer_id IS NULL   THEN 'ID not found'
            WHEN NOT i.in_scope          THEN 'Other portal/project'
            WHEN cv.converted_at IS NULL THEN 'No conversion date'
            ELSE 'OK' END AS check_status,
       COALESCE(a.calls_after_conversion, 0) AS calls_after_conversion
FROM ids i
LEFT JOIN conv cv ON cv.customer_id = i.customer_id
LEFT JOIN agg a   ON a.zynixx_client_id = i.zynixx_client_id;
```

WHY IT WORKS

- `UNNEST(ARRAY[...])` turns the list into rows, in the same format as an IN list.
- LEFT JOINs from the list keep every ID; `check_status` explains each zero.
- Check: rows returned = distinct IDs pasted.

Lesson: When the question is 'for these IDs', start the query from the IDs.

20261102-06 Join a Metabase saved question safely

SQL · **How-to** · {{#3268}}, DISTINCT ON, team list

PROBLEM

Needed agent, TL, manager and team for each call from saved question #3268.

CAUSE

If an agent appears twice in the team list, joining it directly counts each of their calls twice.

FIX

```
team AS (
    SELECT DISTINCT ON (user_id) user_id, agents, tl, manager, team
    FROM {{#3268-team-main}}
    ORDER BY user_id
)
-- check first:
SELECT user_id, COUNT(*) FROM {{#3268-team-main}} GROUP BY user_id HAVING COUNT(*) > 1;
```

WHY IT WORKS

- `{{#3268-...}}` uses a saved question like a table inside a native query.
- `STRING_AGG(DISTINCT agent, ', ' ORDER BY agent)` lists agents once each.
- `(ARRAY_AGG(agent ORDER BY started_at DESC))[1]` is the agent on the latest call.

Lesson: Every lookup table gets a one-row-per-key guard, even a team list.

20261102-07 Keep only the Sales team's calls

SQL · **How-to** · JOIN vs LEFT JOIN, filter on joined table

PROBLEM

The report had to count only calls made by Sales agents.

CAUSE

The team is a column of the joined team list, so the filter belongs on that table.

FIX

```
SELECT team, COUNT(*) AS agents FROM {{#3268-team-main}} GROUP BY team ORDER BY team;  -- exact values

JOIN team t ON t.user_id = cl.agent_user_id
WHERE ... AND t.team = 'Sales'          -- or ILIKE 'sales%' after checking the values
```

WHY IT WORKS

- An inner JOIN plus `t.team = 'Sales'` drops calls by anyone else, including agents not in the list.
- Check the distinct values first, so `ILIKE 'sales%'` doesn't catch `Sales Support`.

Lesson: Look at the real values of a column before filtering on it.

20261102-08 30-day counts beside the day buckets

SQL · **How-to** · FILTER, INTERVAL, cross-check

PROBLEM

Needed calls and answered calls within 30 days of conversion, keeping the existing buckets.

CAUSE

Two more conditional counts in the same aggregation.

FIX

```
COUNT(*)           FILTER (WHERE since_conversion <= INTERVAL '30 days') AS calls_within_30_days,
COUNT(answered_at) FILTER (WHERE since_conversion <= INTERVAL '30 days') AS answered_within_30_days,
```

WHY IT WORKS

- `FILTER` works on any aggregate, including `COUNT(column)`.
- Check: `calls_within_30_days = within_1_day + days_2_to_7 + days_8_to_30` on every row.

> **Lesson:** When two columns must agree, write the check down and run it.

20261102-09 A 30-day request rebuilt three times

Process · **Wrong result** · requirements, output columns

PROBLEM

"Calls within 30 days" became day_0 to day_30 columns, then a stripped query, then a query missing the buckets, before the right version.

CAUSE

It wasn't agreed whether the 30-day columns should replace or be added to the existing ones.

FIX

```
Start from: <the query that's already right>
Add columns: <name> = <definition>, <name> = <definition>
Keep: everything else as it is
One row per: <client / agent / day>
```

WHY IT WORKS

- Naming the exact columns, and what to keep, settles a change in one round.
- "Replace" or "add" is the question to settle before rewriting a working query.

> **Lesson:** Say what to keep, not only what to add.

20261102-10 Admin IDs that share an IP

Metabase · **How-to** · Summarize, distinct count, STRING_AGG, private IPs

PROBLEM

The editor showed how many Admin IDs used each IP, but not which IDs.

CAUSE

Metabase's Summarize has no aggregation that lists values in one cell.

FIX

```
SELECT ip, COUNT(DISTINCT admin_id) AS admin_count,
       STRING_AGG(DISTINCT admin_id::text, ', ' ORDER BY admin_id::text) AS admin_ids
FROM tdx_activity
WHERE ip IS NOT NULL AND ip <> ''
GROUP BY ip
HAVING COUNT(DISTINCT admin_id) > 1
ORDER BY admin_count DESC;
```

WHY IT WORKS

- View the SQL, then Convert this question to SQL, turns the editor question into editable SQL.
- A blank-IP row (10 admins) is missing data, not a shared address.
- `10.x` IPs are private (office, VPN, proxy); IP + user agent is a stronger same-device signal.

Lesson: Know where the editor stops; switch to SQL for lists and complex logic.

Wednesday, 4 November 2026 (covers 3 to 4 Nov)

Focus	Client reports from the ledger (net withdrawals, campaign flag, trading days), a two-sheet broker commission workbook in the MIS design, importing your own modules, and writing functions.
Outcome	11 entries: 2 errors, 3 wrong results and 6 how-tos. The P&L chain became a function at three levels, each checked against the original result.
Review	Repeated: sorting by the key instead of the date before keep='first' (third time). Focus tomorrow: write trading_days(led, start, end) at Level 2 and check it with assert_frame_equal.
Cards	`20261104-01` to `20261104-11`

20261104-01 When a lambda is fine, and when it's 900x slower

pandas · **How-to** · apply, vectorise, benchmark

PROBLEM

Needed a rule for when to use lambdas in pandas.

CAUSE

A lambda inside `.apply` runs once per row in plain Python; one given a whole column or table doesn't.

FIX

```
df.apply(lambda r: r.a + r.b, axis=1)     # 4,718 ms on 500,000 rows
df.a + df.b                               #     5 ms

calls.assign(talk_s=lambda d: d['Talk Time'].dt.total_seconds())   # fine: d is the whole table
max(files, key=lambda f: f.stat().st_mtime)                        # fine: key= function
```

WHY IT WORKS

- Order to try: column operators, np.where/np.select/pd.cut, groupby built-ins, map/str, then apply.
- Surprise on pandas 3: .str.split('|').str[1] was the slowest text method; .str.replace(regex) the fastest.
- Joining text per group (', '.join) has no built-in, so a lambda is right there.

Lesson: Give a lambda whole columns, never one row at a time.

20261104-02 Deposits and withdrawals net of cancellations

pandas · **How-to** · where, groupby, ledger types

PROBLEM

Needed Deposit and Withdraw per client, with WITHDRAW CANCELLED credits subtracted.

CAUSE

The pivot_table used two lambdas that looked rows up again with .loc[x.index].

FIX

```
lt = req_led['LedgerType'].str.strip().str.upper()
cr, dr = req_led['Credit'].fillna(0), req_led['Debit'].fillna(0)
one_pivot = (req_led
    .assign(Deposit=cr.where(lt.eq('DEPOSIT'), 0),
            Withdraw=dr.where(lt.eq('WITHDRAW'), 0) - cr.where(lt.eq('WITHDRAW CANCELLED'), 0))
    .groupby('ClientID', as_index=False)[['Deposit', 'Withdraw']].sum())
```

WHY IT WORKS

- Each row contributes +debit, -credit or 0, so the sum is withdrawals minus cancellations.
- fillna(0) first: one blank Credit would make the row blank and sum would skip it.

> **Lesson:** Build the amount columns, then one plain sum.

20261104-03 np.where got one argument: a bracket closed too late

pandas · **Error** · np.where, &, brackets

PROBLEM

`np.where((df['Balance'] == 0) & (df['Last Trade'] != 'No Trade', 'Yes', 'No'))` failed.

CAUSE

The second condition's `)` closed after 'No', so Python built a tuple and np.where got one argument.

FIX

```
df['Traded Zero'] = np.where(
    (df['Balance'] == 0) & (df['Last Trade'] != 'No Trade'),
    'Yes', 'No')
```

WHY IT WORKS

- Each condition in its own brackets; & | ~ between them; then the true and false values.
- `and` / `or` raise 'truth value of a Series is ambiguous'.
- 0.004 isn't == 0: use .round(2).eq(0) when tiny balances should count.

> **Lesson:** Close every condition's bracket before the first comma.

20261104-04 Latest balance taken from a random row (again)

pandas · **Wrong result** · sort_values, drop_duplicates, repeated

PROBLEM

`balance = req_led.sort_values('ClientID').drop_duplicates('ClientID', keep='first').`

CAUSE

Sorting by the key leaves rows within each client in an undefined order, so 'first' isn't the latest.

FIX

```
balance = (req_led.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first')
                  [['ClientID', 'CreatedAt', 'Balance']])
```

WHY IT WORKS

- Third time in the journal (29 Oct, 30 Oct, 3 Nov).
- Balance feeds Traded Zero, so one wrong sort spreads into two columns.

> **Lesson:** Before keep='first', the line above must sort by a date.

20261104-05 Campaign flag: 'deposited on' vs 'first deposit was'

pandas · **Wrong result** · first row per group, requirements

PROBLEM

The flag was built for any 1,000 deposit on 25 Aug; the rule was the client's FIRST deposit.

CAUSE

Every deposit was tested instead of only each client's earliest one.

FIX

```
first_dep = dep.sort_values('CreatedAt').drop_duplicates('ClientID', keep='first')
camp_ids = set(first_dep.loc[(first_dep['CreatedAt'].dt.normalize() == pd.Timestamp('2026-08-25'))
                             & (first_dep['Credit'] == 1000), 'ClientID'])
df['Campaign'] = np.where(df['ClientID'].isin(camp_ids), 'Campaign', 'Not Campaign')
```

WHY IT WORKS

- 'First X': sort oldest first, keep the first row per client, then test it.
- 'Only X': also require one deposit in total.
- First within the loaded ledger: check led['CreatedAt'].min().

Lesson: Pin down first / any / only before writing the filter.

20261104-06 Trading days and their spread per client

pandas · **How-to** · nunique, join per group, to_period

PROBLEM

Needed trading days per client, plus the dates and a per-month count as text.

CAUSE

Several bills per day must count once; month labels must stay in calendar order.

FIX

```
bills = (req_led[lt.eq('BILL')].assign(Trade_Date=lambda d: d['CreatedAt'].dt.normalize())
           .drop_duplicates(['ClientID', 'Trade_Date']).sort_values(['ClientID', 'Trade_Date']))
dates = bills.groupby('ClientID')['Trade_Date'].agg(lambda s: ', '.join(s.dt.strftime('%d %b')))
per_month = (bills.assign(M=lambda d: d['Trade_Date'].dt.to_period('M'))
                  .groupby(['ClientID', 'M']).size().reset_index(name='n')
                  .assign(t=lambda d: d['M'].dt.strftime('%b %Y') + ': ' + d['n'].astype(str))
                  .groupby('ClientID')['t'].agg(', '.join))
```

WHY IT WORKS

- drop_duplicates on client + date keeps one row per trading day.
- Sorting by the Period, not the text, keeps months in calendar order.
- Check: days per month add up to Trading Days; 0 days where Last Trade is 'No Trade'.

Lesson: Join text per group with ', '.join after sorting.

20261104-07 Excel export in the MIS design

pandas · **How-to** · openpyxl, ExcelWriter, number_format

PROBLEM

The commission table had to look like the MIS file: coloured bands, gold headers, formats.

CAUSE

pandas writes plain cells; styling needs openpyxl on the sheet pandas just wrote.

FIX

```
with pd.ExcelWriter(path, engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Brokers', index=False, startrow=1)   # row 1 left for bands
    ws = writer.sheets['Brokers']
    ws.merge_cells(start_row=1, start_column=1, end_row=1, end_column=4)
    ws.cell(1, 1).value = 'Admin Panel Broker File'
    for row in ws.iter_rows(min_row=3):
        for cell in row[4:]:
            cell.number_format = '#,##0.00;[Red]-#,##0.00;"-"'
    ws.freeze_panes, ws.auto_filter.ref = 'B3', f"A2:K{ws.max_row}"
```

WHY IT WORKS

- startrow=1 leaves Excel row 1 free; writer.sheets[name] is the openpyxl sheet.
- Number format parts: positive; negative (red); zero (dash).
- One style_sheet() function styles both the Brokers and the Clients sheets.

Lesson: Write with pandas, style with openpyxl, in the same with-block.

20261104-08 NBP always 0: rows filtered away one step early

pandas · **Wrong result** · filter order, isin

PROBLEM

The broker sheet showed 0 NBP for every client.

CAUSE

led_oct_bill kept only BILL rows, so the later filter for BILL + NBP had no NBP rows left.

FIX

```
lt = ['BILL', 'NEGATIVE BALANCE PROTECTION']
led_child = led_oct[led_oct['ClientID'].isin(child)]          # not led_oct_bill
led_child_fil = led_child[led_child['LedgerType'].isin(lt)]
```

WHY IT WORKS

- Check after each filter: df['LedgerType'].value_counts() shows which types survived.
- Same broker rule: .where(is_loss, 'TEXT') keeps numbers numeric; np.where would turn them into text.

Lesson: A filter can't keep rows an earlier filter already removed.

20261104-09 TypeError: insert expected 2 arguments, got 1

Python · **Error** · sys.path, imports, PYTHONPATH

PROBLEM

`sys.path.insert(r'C:\...\migration')` failed while importing ledger_loader from another folder.

CAUSE

list.insert needs a position and an item; there's no default position.

FIX

```
import sys
sys.path.insert(0, r"C:\Users\Administrator\Videos\Office\migration")   # 0 = searched first
from ledger_loader import get_final_ledger
# permanent: setx PYTHONPATH "C:\Users\Administrator\Videos\Office\migration", then restart VS Code
```

WHY IT WORKS

- Import checks sys.modules first, then searches sys.path from position 0; the first match wins.
- append(folder) takes one argument but searches the folder last.
- module.__file__ shows which file was imported; %autoreload 2 picks up edits.

Lesson: insert(0, folder) puts your folder first in the search.

20261104-10 Writing functions in three levels

Python · **How-to** · def, parameters, defaults, None, raise

PROBLEM

Needed to turn the P&L chain into a reusable function, starting as a beginner.

CAUSE

Copying cells between notebooks repeats work and spreads mistakes.

FIX

```
def client_pnl_v1(led):                                          # level 1: wrap working code
    return led.assign(...).groupby(...).sum().assign(...)

def client_pnl_v2(led, start, end, commission_rate=0.05):          # level 2: parameters + default
    rows = led[(led['CreatedAt'] >= start) & (led['CreatedAt'] < end)]
    ...

def client_pnl_v3(led, start=None, end=None, clients=None, commission_rate=0.05):   # level 3
    if missing: raise ValueError(...)
    if clients is not None: rows = rows[rows['ClientID'].isin(clients)]
```

WHY IT WORKS

- def runs nothing; calling the function does. return sends one result out.
- Names inside a function stay inside; only `x = f(...)` outside creates a notebook variable.
- None as a default means 'not given'; raise ValueError stops early with a clear message.

Lesson: Wrap working code first, then turn fixed values into parameters one at a time.

20261104-11 Prove a rewrite didn't change the numbers

pandas · **How-to** · assert_frame_equal, testing

PROBLEM

Needed to confirm each function version matches the existing pnl table.

CAUSE

Looking at a few rows can't prove two tables are identical.

FIX

```
pd.testing.assert_frame_equal(pnl.reset_index(drop=True),
                              client_pnl_v1(led_child_fil).reset_index(drop=True),
                              check_dtype=False)
```

WHY IT WORKS

- Silent when equal; raises an error showing the first difference when not.
- Level 3 is expected to differ: it reads the full ledger, so NBP rows come back in.

Lesson: Every rewrite gets an equality check against the version you trust.

Template: day page

Copy this page for each new day. Fill Focus in the morning and Outcome before you log off.

Date	
Focus	
Outcome	
Review	Repeated mistakes, gaps, one focus for tomorrow
Cards	_____ to _____

Index lines to add tonight

ID	Tool	Type	Title	Tags

End-of-day check

- ☐ Every solved problem has a card
- ☐ Error messages pasted word for word
- ☐ Cards added to the Index
- ☐ Any lesson seen before? Add it to Patterns

Template: log card

One card per problem. Two cards fit on a page when the fix is short.

ID		Tool		Type	Error / Wrong result / How-to
Title					
Tags					
Problem					
Cause					
Fix					
Why it works					
Lesson					

ID		Tool		Type	Error / Wrong result / How-to
Title					
Tags					
Problem					
Cause					
Fix					
Why it works					
Lesson					